

## Introduction

### Warning during vLLM startup

```
vllm serve Qwen/Qwen3.8-27B-FP8
```

```
...
```

```
WARNING [fp8_utils.py:851] Using default W8A8 Block FP8 kernel config. Performance might be sub-optimal!
```

```
Config file not found at .../configs/N=17408,K=5120,device_name=NVIDIA_L40S,dtype=fp8_w8a8,block_shape=[128,128].json
```

```
...
```

## Introduction

## Matrix multiplication

$$A \cdot B = C$$

$$\begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 3 \end{bmatrix} \cdot \begin{bmatrix} 2 & 1 \\ 0 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 2 & 3 \\ 3 & 1 \end{bmatrix}$$

## Introduction

### Matrix multiplication shape

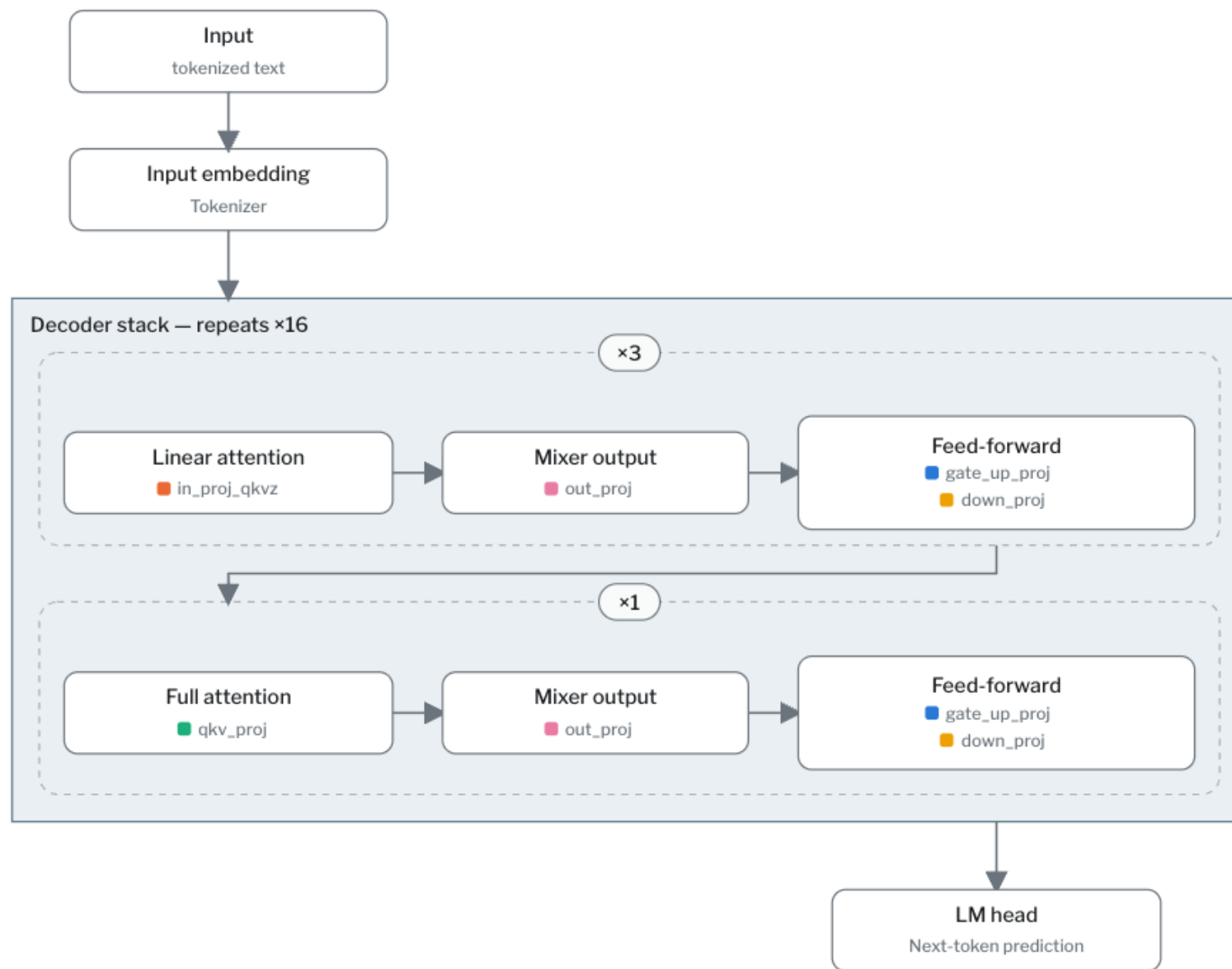
$$A \cdot B = C$$

$$M \left\{ \begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 3 \end{bmatrix} \right. \cdot K \left\{ \underbrace{\begin{bmatrix} 2 & 1 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}}_N \right. = M \left\{ \underbrace{\begin{bmatrix} 2 & 3 \\ 3 & 1 \end{bmatrix}}_N \right.$$

Here,  $M=2$  tokens,  $K=3$  input features,  $N=2$  output features.

## Introduction

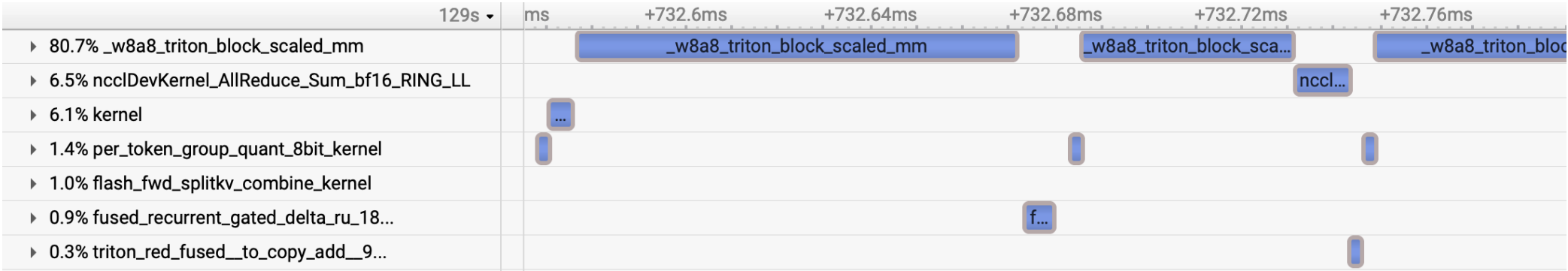
### Qwen3.8-27B architecture



- in\_proj\_qkvz N=8192, K=5120
- out\_proj N=5120, K=3072
- gate\_up\_proj N=17408, K=5120
- down\_proj N=5120, K=8704
- qkv\_proj N=7168, K=5120

Introduction

Use NVIDIA Nsight Systems to check kernel launches



## Introduction

# How is the matrix multiplication implemented?

Using Triton in `vllm/model_executor/layers/quantization/utils/fp8_utils.py`:

```
@triton.jit
def _w8a8_triton_block_scaled_mm(
    # Pointers to inputs and output
    A,
    B,
    C,
    As,
    Bs,
    # Shape for matmul
    M,
    N,
    K,
    # Block size for block-wise quantization
    group_n,
    group_k,
    # Stride for inputs and output
    stride_am,
    stride_ak,
    stride_bk,
    stride_bn,
    stride_cm,
    stride_cn,
    stride_as_m,
    stride_as_k,
    stride_bs_k,
    stride_bs_n,
    # Meta-parameters
    BLOCK_SIZE_M: tl.constexpr,
    BLOCK_SIZE_N: tl.constexpr,
    BLOCK_SIZE_K: tl.constexpr,
    GROUP_SIZE_M: tl.constexpr,
):
    """Triton-accelerated function used to perform linear operations (dot
    product) on input tensors 'A' and 'B' with block-wise quantization, and
    store the result in output tensor 'C'.
    """
    pid = tl.program_id(axis=0)
    num_pid_m = tl.cdiv(M, BLOCK_SIZE_M)
    num_pid_n = tl.cdiv(N, BLOCK_SIZE_N)
    num_pid_in_group = GROUP_SIZE_M * num_pid_n
    group_id = pid // num_pid_in_group
    first_pid_m = group_id * GROUP_SIZE_M
    group_size_m = min(num_pid_m - first_pid_m, GROUP_SIZE_M)
    pid_m = first_pid_m + (pid % group_size_m)
    pid_n = (pid % num_pid_in_group) // group_size_m

    offs_am = (pid_m * BLOCK_SIZE_M + tl.arange(0, BLOCK_SIZE_M)) % M
    offs_bn = (pid_n * BLOCK_SIZE_N + tl.arange(0, BLOCK_SIZE_N)) % N
    offs_k = tl.arange(0, BLOCK_SIZE_K)
    a_ptrs = A + (offs_am[:, None] * stride_am + offs_k[None, :] * stride_ak)
    b_ptrs = B + (offs_k[:, None] * stride_bk + offs_bn[None, :] * stride_bn)

    As_ptrs = As + offs_am * stride_as_m
    offs_bsn = offs_bn // group_n
    Bs_ptrs = Bs + offs_bsn * stride_bs_n

    accumulator = tl.zeros((BLOCK_SIZE_M, BLOCK_SIZE_N), dtype=tl.float32)
    for k in range(0, tl.cdiv(K, BLOCK_SIZE_K)):
        a = tl.load(a_ptrs, mask=offs_k[None, :] < K - k * BLOCK_SIZE_K, other=0.0)
        b = tl.load(b_ptrs, mask=offs_k[:, None] < K - k * BLOCK_SIZE_K, other=0.0)

        k_start = k * BLOCK_SIZE_K
        offs_ks = k_start // group_k
        a_s = tl.load(As_ptrs + offs_ks * stride_as_k)
        b_s = tl.load(Bs_ptrs + offs_ks * stride_bs_k)

        accumulator += tl.dot(a, b) * a_s[:, None] * b_s[None, :]
        a_ptrs += BLOCK_SIZE_K * stride_ak
        b_ptrs += BLOCK_SIZE_K * stride_bk

    if C.dtype.element_ty == tl.bfloat16:
        c = accumulator.to(tl.bfloat16)
    elif C.dtype.element_ty == tl.float16:
        c = accumulator.to(tl.float16)
    else:
        c = accumulator.to(tl.float32)

    offs_cm = pid_m * BLOCK_SIZE_M + tl.arange(0, BLOCK_SIZE_M)
    offs_cn = pid_n * BLOCK_SIZE_N + tl.arange(0, BLOCK_SIZE_N)
    c_ptrs = C + stride_cm * offs_cm[:, None] + stride_cn * offs_cn[None, :]
    c_mask = (offs_cm[:, None] < M) & (offs_cn[None, :] < N)
    tl.store(c_ptrs, c, mask=c_mask)
```

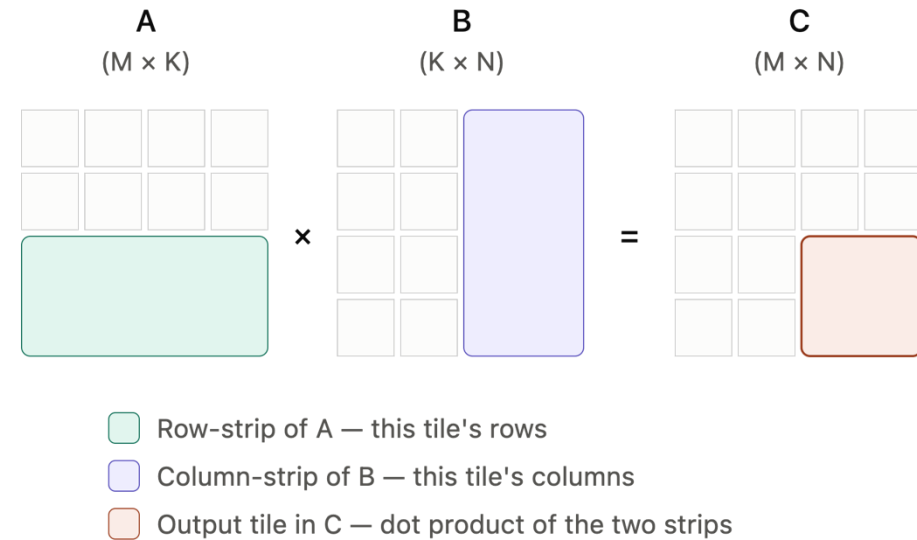
## Introduction

### The kernel params

By default:

```
{  
  "BLOCK_SIZE_M": 64,  
  "BLOCK_SIZE_N": 128,  
  "BLOCK_SIZE_K": 128,  
  "GROUP_SIZE_M": 32,  
  "num_warps": 4,  
  "num_stages": 2  
}
```

What are blocks/tiles?



Introduction

Tuned configs for certain GEMM shape + GPU model combinations

vllm / vllm / model\_executor / layers / quantization / utils / configs /

| Name                                                                      | Last commit message                                                  | Last commit date |
|---------------------------------------------------------------------------|----------------------------------------------------------------------|------------------|
| ..                                                                        |                                                                      |                  |
| N=1024,K=2048,device_name=AMD_Instinct_MI300X,dtype=fp8_w8a8,block_sh...  | [AMD][QWEN3-NEXT] FP8 Tunings (#32042)                               | 8 months ago     |
| N=10240,K=5120,device_name=NVIDIA_L40S,dtype=fp8_w8a8,block_shape=[12...  | [Model][Qwen3VL] Tune Triton w8a8 block fp8 kernel for L40s (#29217) | 10 months ago    |
| N=12288,K=2048,device_name=AMD_Instinct_MI300X,dtype=fp8_w8a8,block_s...  | [AMD][QWEN3-NEXT] FP8 Tunings (#32042)                               | 8 months ago     |
| N=12288,K=7168,device_name=NVIDIA_H100_80GB_HBM3,dtype=fp8_w8a8,blo...    | [Perf] Tune configs for triton block fp8 gemm H100/H200 (#23748)     | last year        |
| N=12288,K=7168,device_name=NVIDIA_H200,dtype=fp8_w8a8,block_shape=[12...  | [Perf] Tune configs for triton block fp8 gemm H100/H200 (#23748)     | last year        |
| N=1536,K=1536,device_name=AMD_Instinct_MI300X,dtype=fp8_w8a8,block_sha... | [AMD] [Model] DeepSeek tunings (#13199)                              | last year        |
| N=1536,K=1536,device_name=AMD_Instinct_MI325X,dtype=fp8_w8a8,block_sha... | MI325 configs, fused_moe_kernel bugfix (#14987)                      | last year        |
| N=1536,K=1536,device_name=AMD_Instinct_MI325,dtype=fp8_w8a8,block...      | MI325 configs, fused_moe_kernel bugfix (#14987)                      | last year        |

But missing for our model’s shapes on L40S:

```
vllm serve Qwen/Qwen3.8-27B-FP8
...
WARNING [fp8_utils.py:851] Using default W8A8 Block FP8 kernel config. Performance might be sub-optimal!
Config file not found at .../configs/N=17408,K=5120,device_name=NVIDIA_L40S,dtype=fp8_w8a8,block_shape=[128,128].json
...
```

## Autotuning

Let's use vLLM's autotuning script to get custom configs!

```
vllm / benchmarks / kernels / benchmark_w8a8_block_fp8.py 

Code Blame 416 lines (353 loc) · 11.9 KB

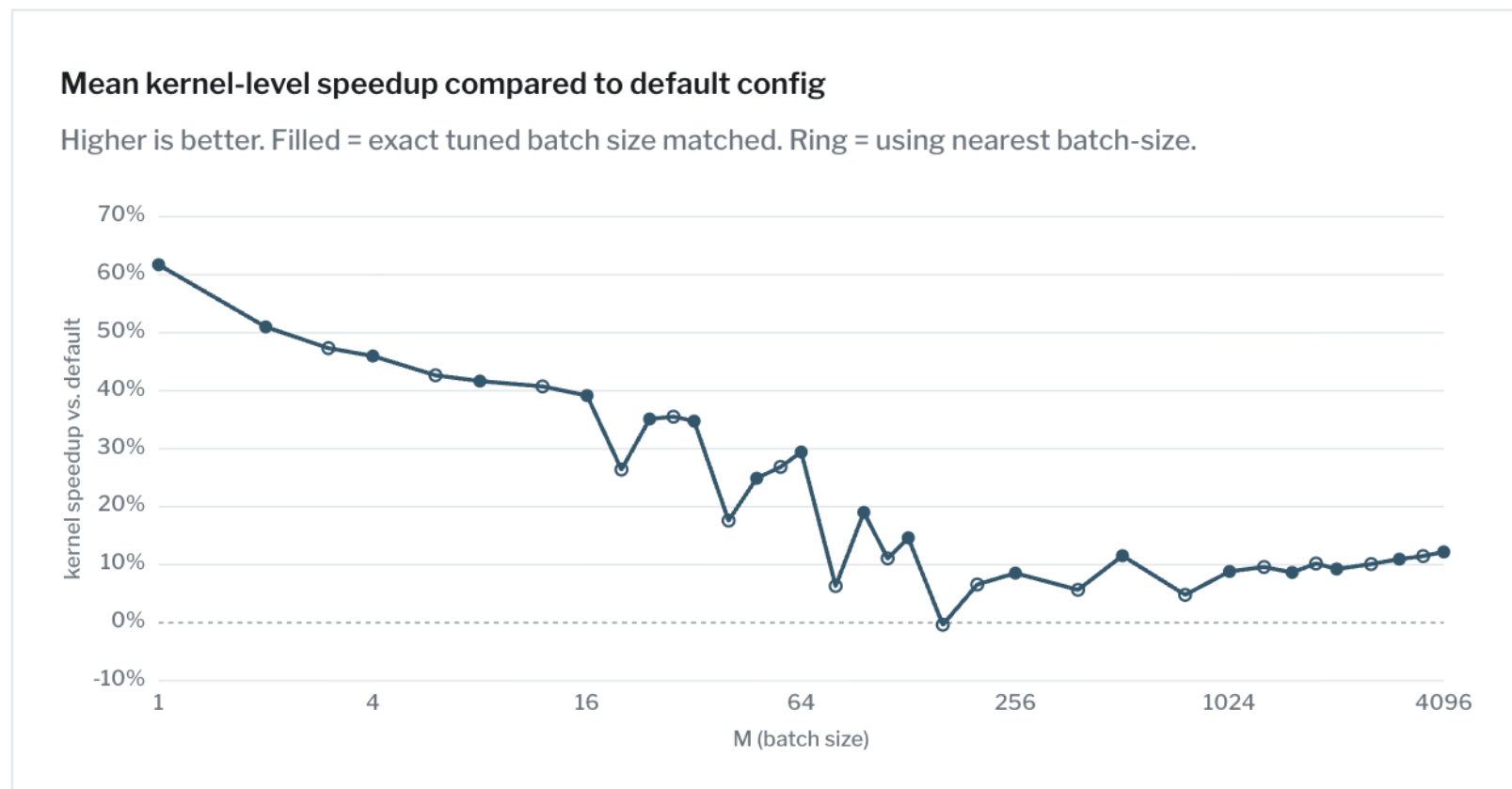
117
118 ✓ def get_configs_compute_bound():
119     configs = []
120     for num_stages in [2, 3, 4, 5]:
121         for block_m in [16, 32, 64, 128, 256]:
122             for block_k in [64, 128]:
123                 for block_n in [32, 64, 128, 256]:
124                     for num_warps in [4, 8]:
125                         for group_size in [1, 16, 32, 64]:
126                             configs.append(
127                                 {
128                                     "BLOCK_SIZE_M": block_m,
129                                     "BLOCK_SIZE_N": block_n,
130                                     "BLOCK_SIZE_K": block_k,
131                                     "GROUP_SIZE_M": group_size,
132                                     "num_warps": num_warps,
133                                     "num_stages": num_stages,
134                                 }
135                             )
136     return configs
137
```



```
#N=17408,K=5120,device_name=NVIDIA_L40S,dtypes=fp8_w8a8,block_shape=[128,128].json
{
  "1": {
    "BLOCK_SIZE_M": 16,
    "BLOCK_SIZE_N": 64,
    "BLOCK_SIZE_K": 128,
    "GROUP_SIZE_M": 32,
    "num_warps": 8,
    "num_stages": 4
  },
  "2": {
    "BLOCK_SIZE_M": 16,
    "BLOCK_SIZE_N": 64,
    "BLOCK_SIZE_K": 128,
    "GROUP_SIZE_M": 64,
    "num_warps": 8,
    "num_stages": 3
  },
  "4": {
    "BLOCK_SIZE_M": 16,
    "BLOCK_SIZE_N": 64,
    "BLOCK_SIZE_K": 128,
    "GROUP_SIZE_M": 32,
    "num_warps": 8,
    "num_stages": 5
  },
  ...
}
```

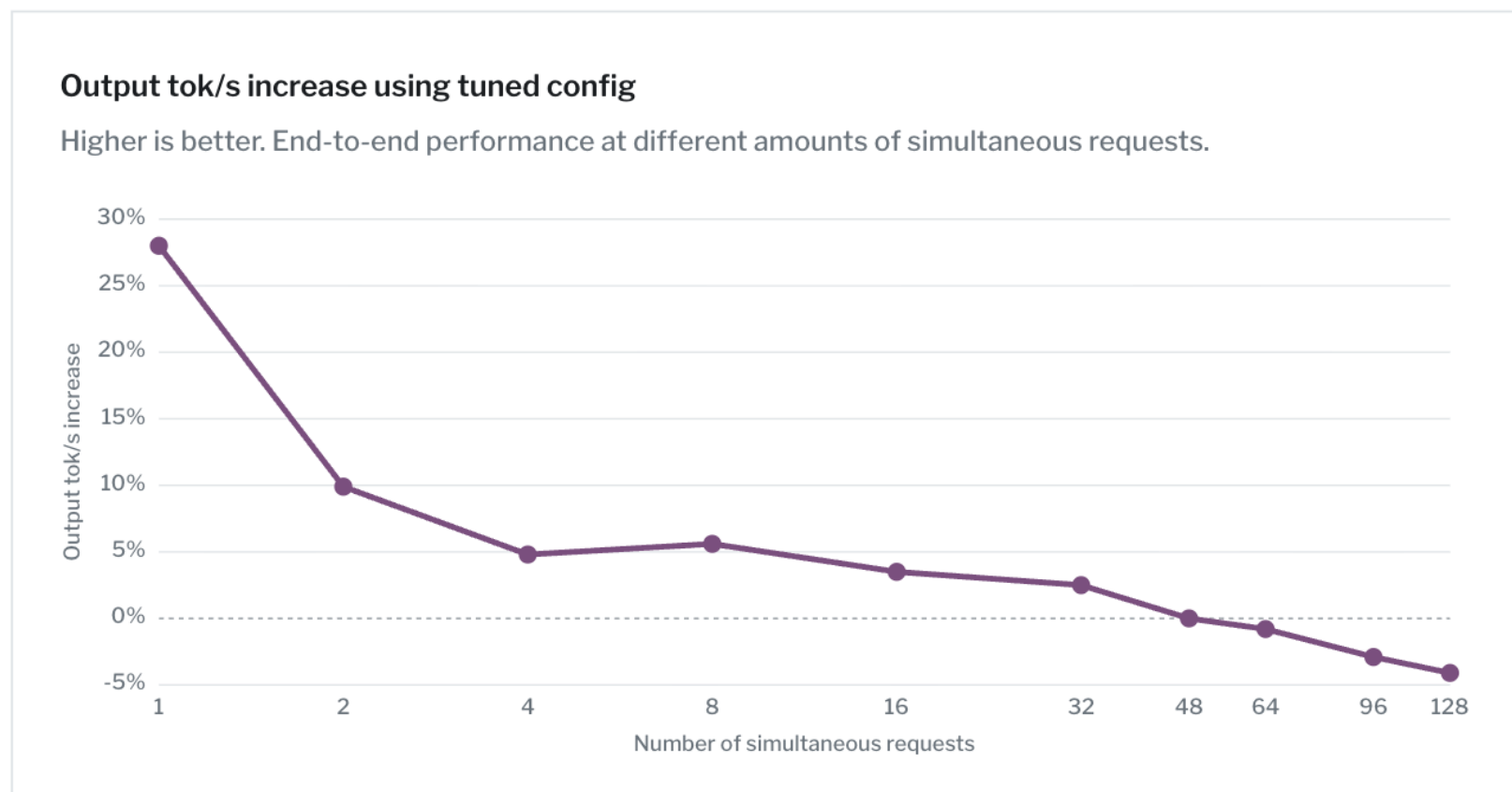
Use autotuning script

How much does the performance improve?



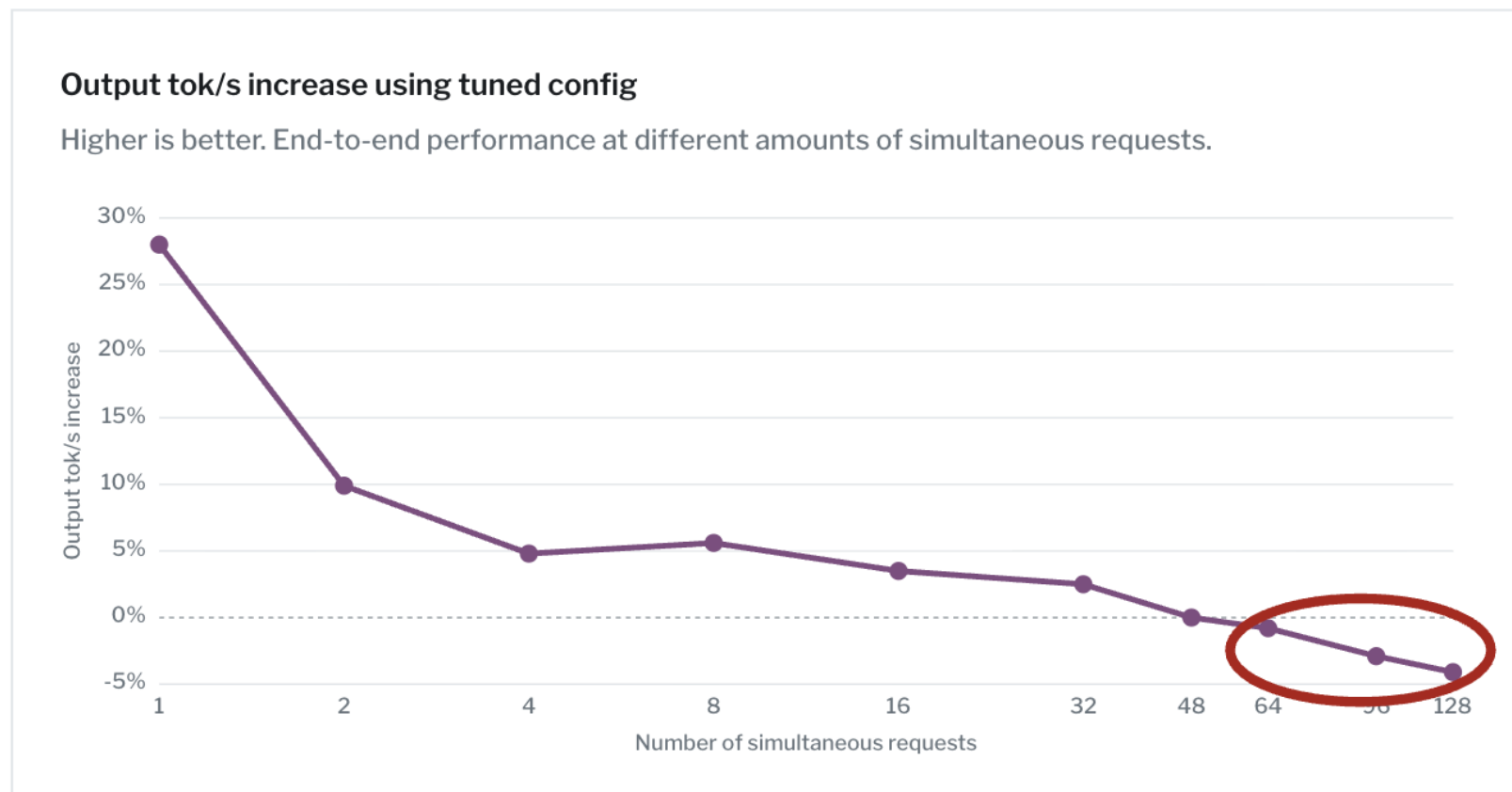
Use autotuning script

How much does the performance improve?



Use autotuning script

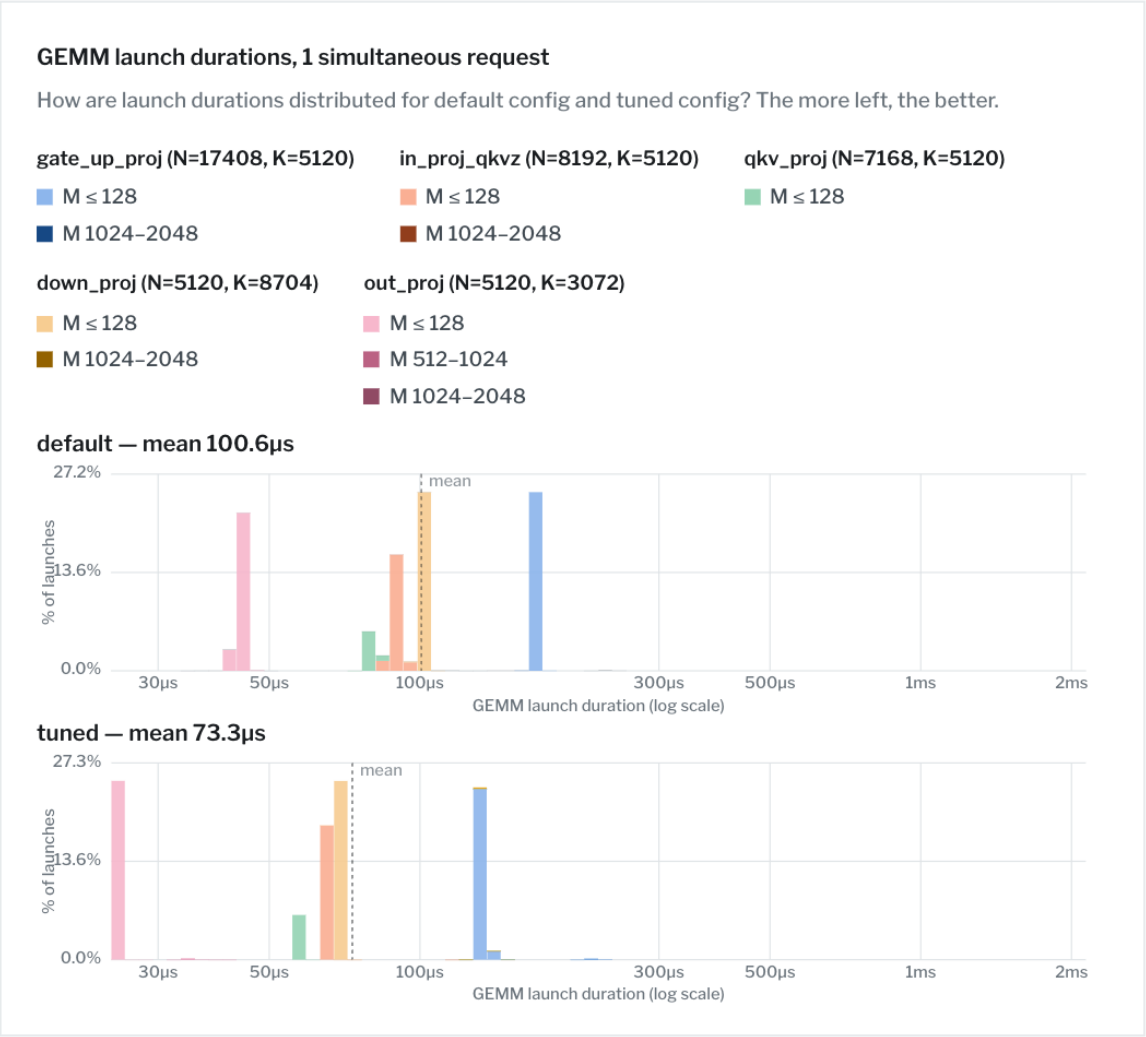
How much does the performance improve?



How can a kernel config that's faster across the whole batch-size range make the e2e system slower under high-concurrency load?

Reason for high-concurrency performance worsening

How does it look like for 1 simultaneous request?



Reason for high-concurrency performance worsening

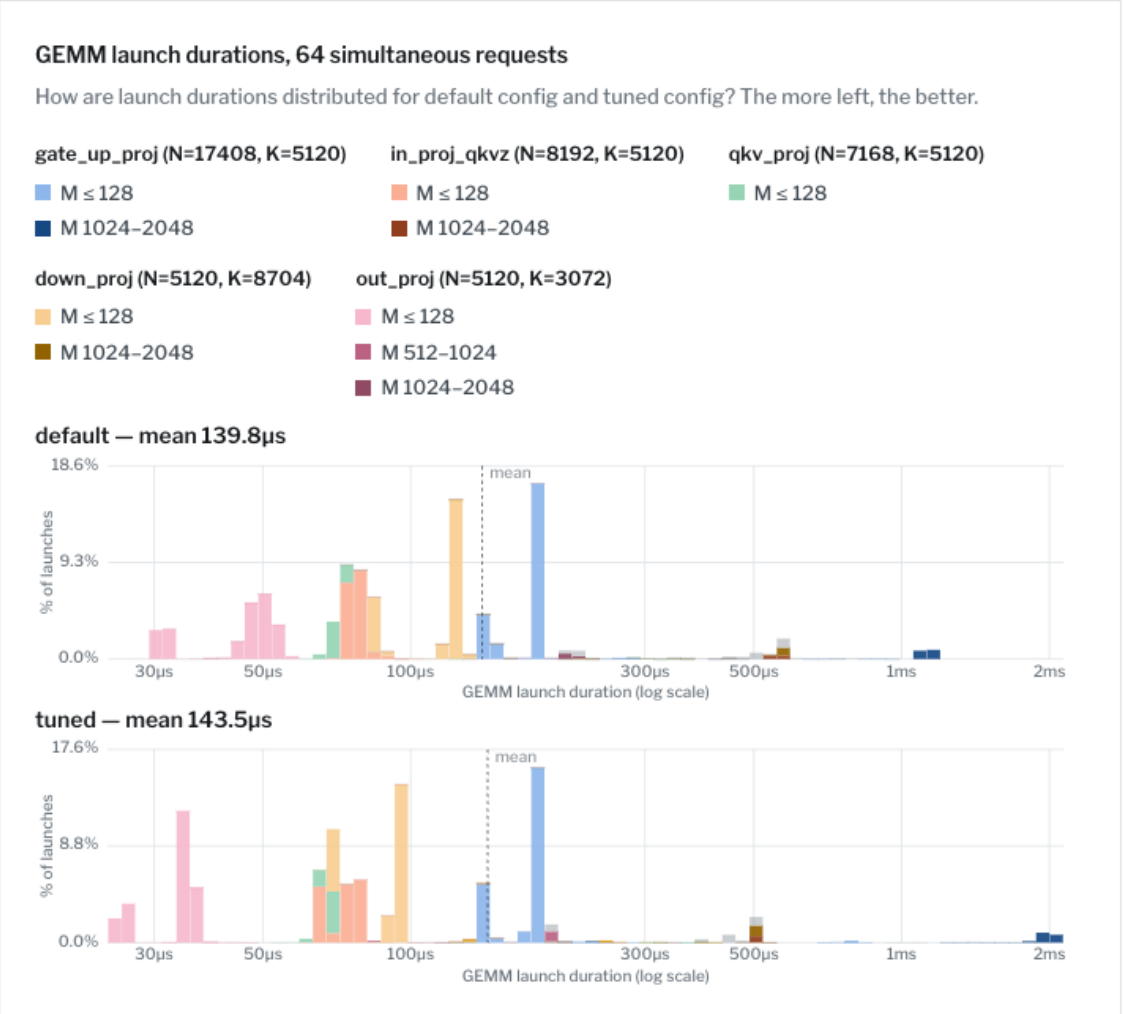
How does it look like for 1 simultaneous request?



=> Looks good.

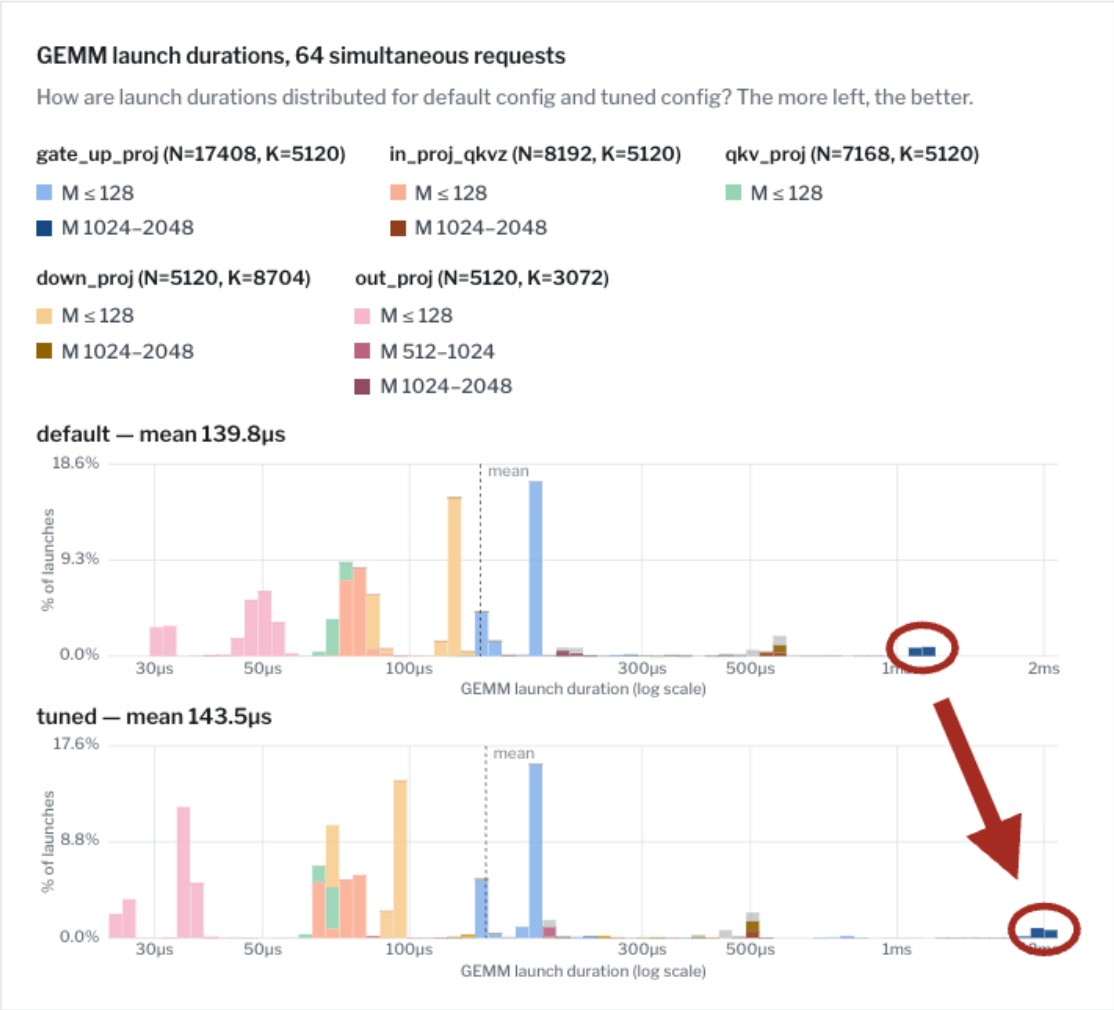
Reason for high-concurrency performance worsening

How does it look like for 64 simultaneous request?



Reason for high-concurrency performance worsening

How does it look like for 64 simultaneous request?



=> Bad. Why is the cluster with shape  $M \approx 2000$ ,  $N=17408$ ,  $K=5120$  taking almost double the time?

Reason for high-concurrency performance worsening

Could it be related to DRAM access?

| Benchmark    | Tensor Active | DRAM Read |
|--------------|---------------|-----------|
| Isolated     | 58.2%         | 14.9%     |
| Real serving | 28.3%         | 72.8%     |

=> Yes. But why is more data pulled from DRAM under real serving than in isolation?

Reason for high-concurrency performance worsening

The autotuning script get's the matrices from L2 cache

```
A, As = make_A(M, K)
B, Bs = make_B(N, K)          # allocated ONCE
for config in search_space:
    t = benchmark_config(A, B, As, Bs, config)  # every launch reuses the SAME B
```

BUT: During real inferencing, there are many other things which go through the cache before the same layer gets called again using the same weight matrix B.

=> B will have been evicted from L2 cache.

The autotuner measures a GEMM whose operands are cached.  
Real serving never lets the operands stay in cache.

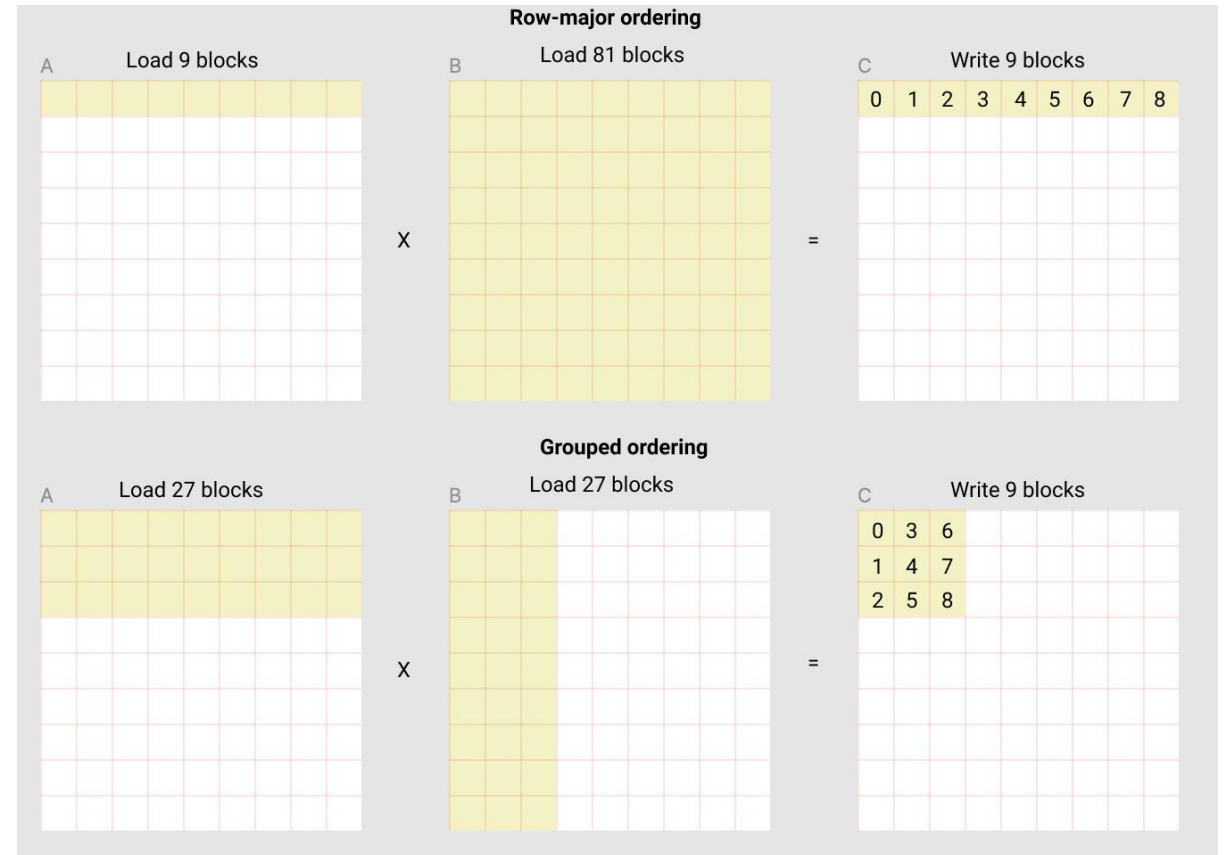
Narrowing down the effect

Which param gets the most bad influence from the wrong cache assumption?

```
"2048": {  
  "BLOCK_SIZE_M": 128,  
  "BLOCK_SIZE_N": 128,  
  "BLOCK_SIZE_K": 128,  
  "GROUP_SIZE_M": 1,  
  "num_warps": 8,  
  "num_stages": 3  
},
```

“BLOCK\_SIZE\_M (...) affects the L2 cache hit rate”

- [Triton Matrix Multiplication Tutorial](#)



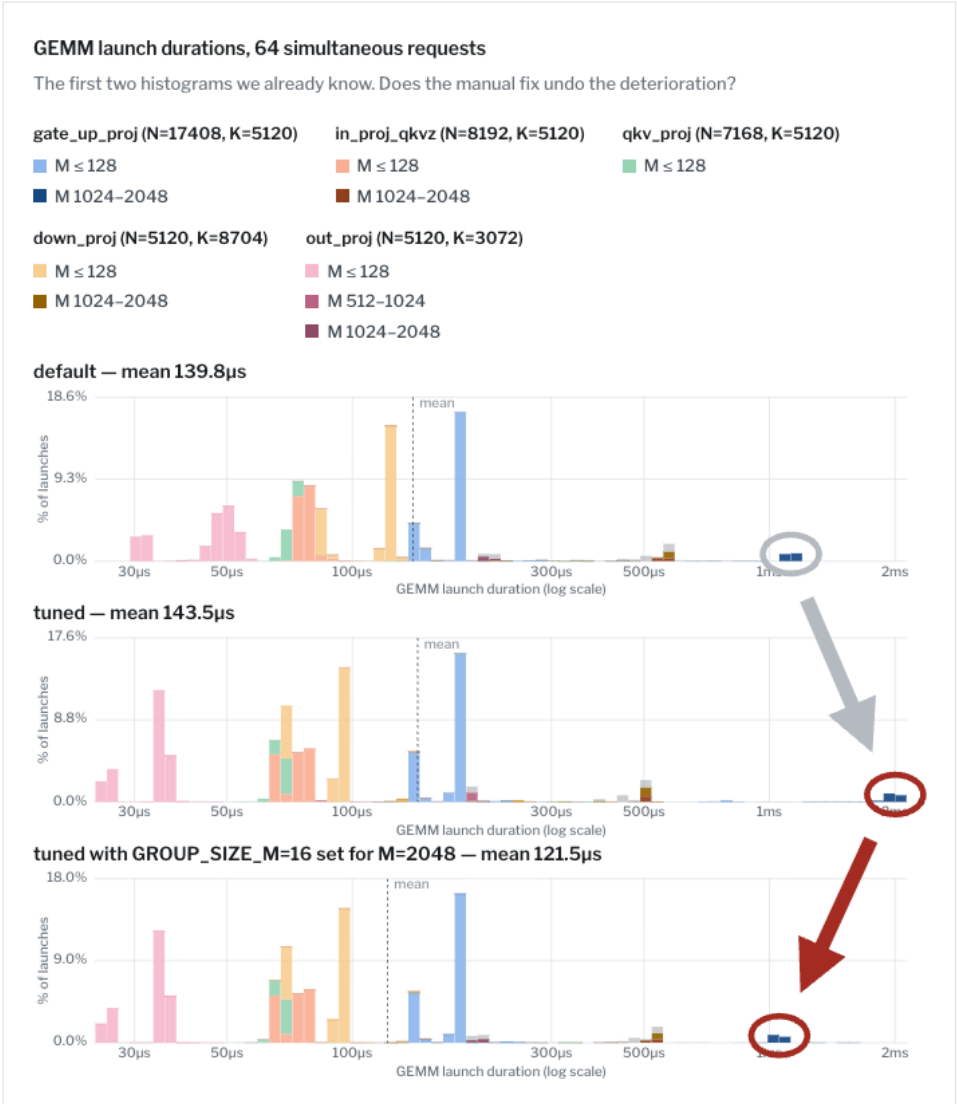
## Narrowing down the effect

## Does changing GROUP\_SIZE\_M from 1 to 16 fix it?



Narrowing down the effect

Does changing GROUP\_SIZE\_M from 1 to 16 fix it?



=> Yes!

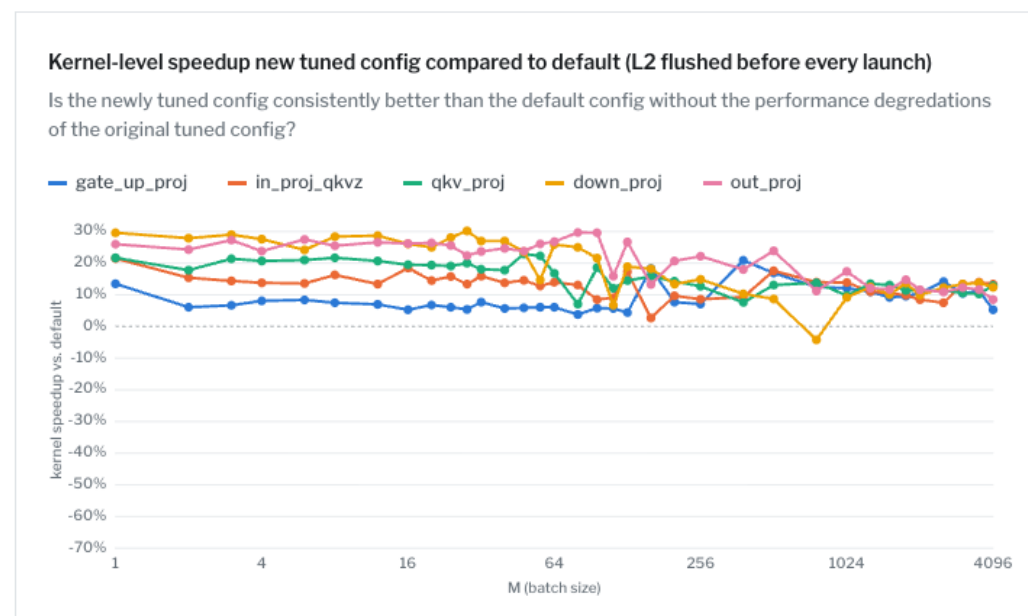
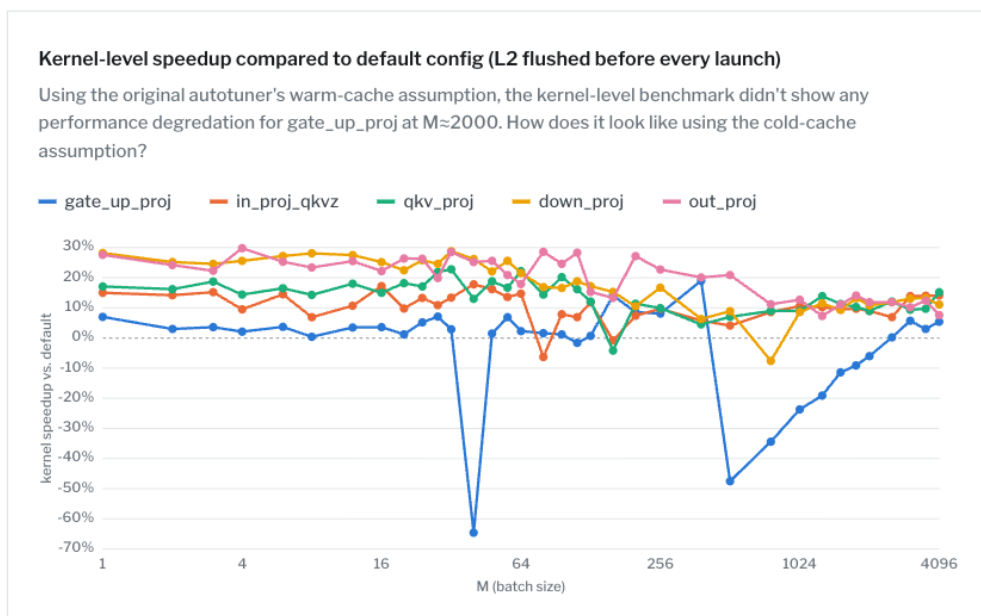
Fixing the autotune script

**Solution: Evict cache before every run**

```
def benchmark_config(...):  
    for _ in range(num_iters):  
        flush_buf.zero_()           # the fix: evict L2 before every launch  
        start_event.record()  
        run()  
        end_event.record()
```

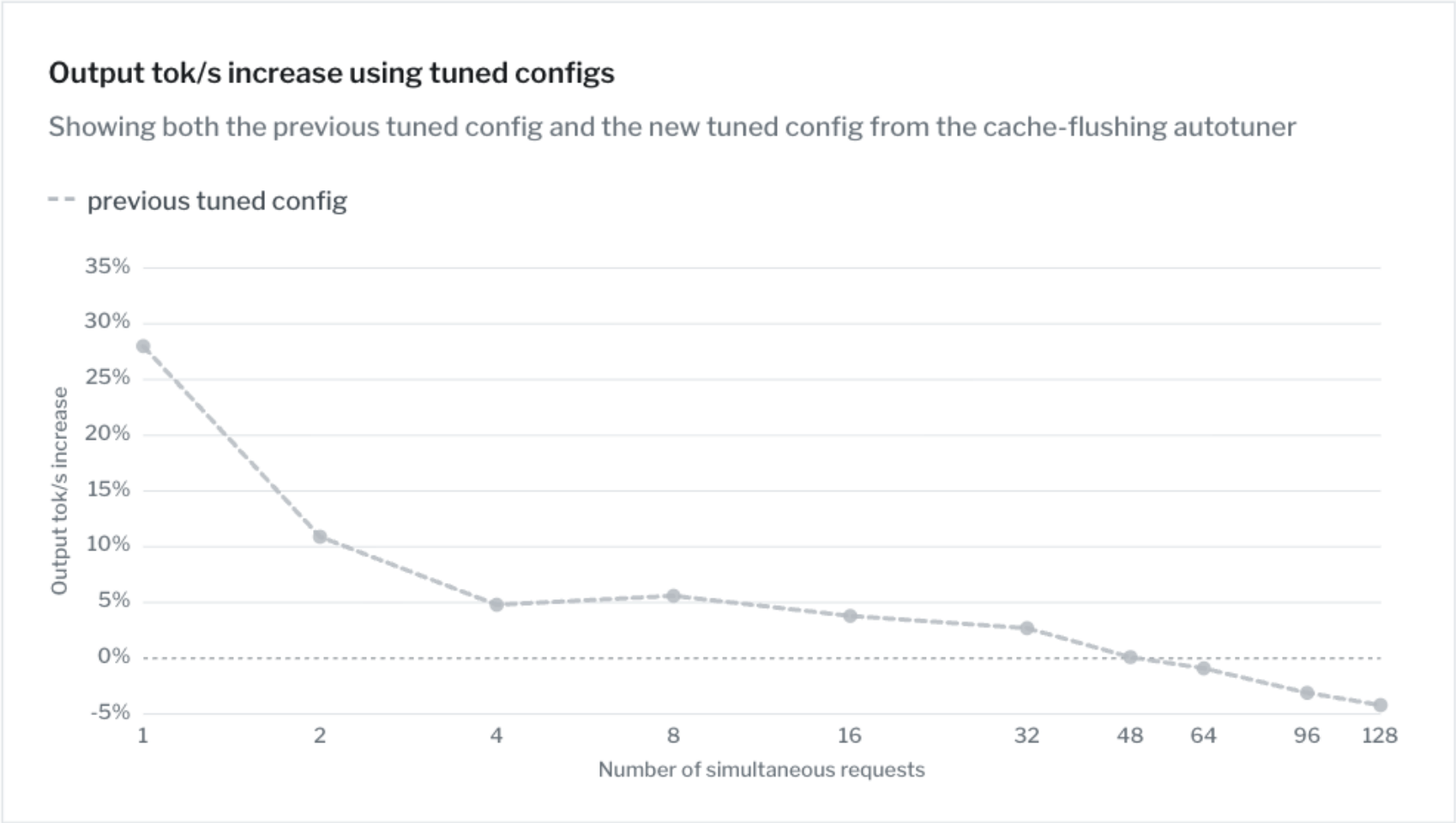
Fixing the autotune script

Before vs. after fix (both measured with L2 flushed before every iteration)



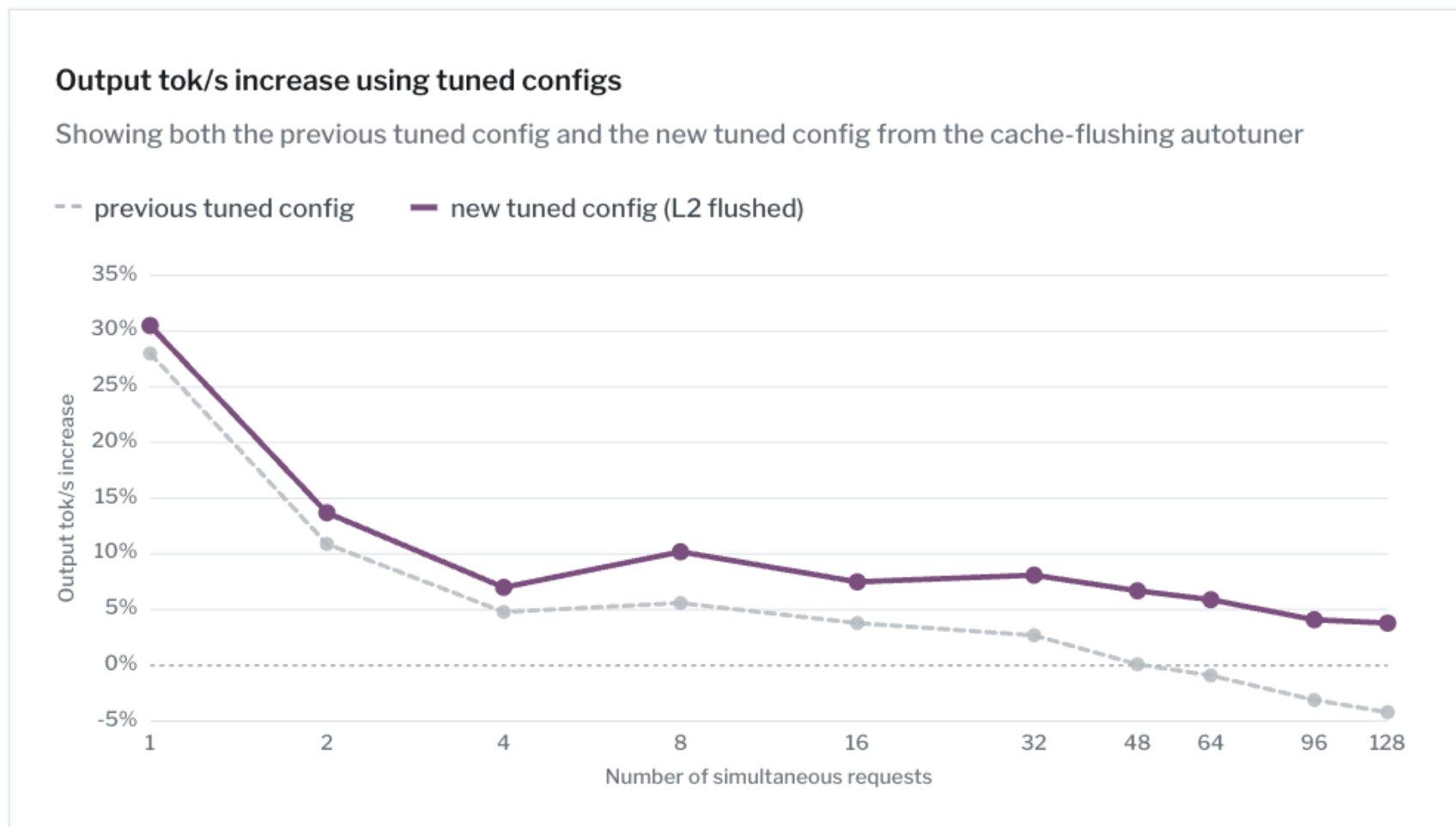
Fixing the autotune script

# Real serving performance improvement



Fixing the autotune script

## Real serving performance improvement



=> Better across  
all concurrencies!